Subject outline

Lecture topics (by week)

1 – Introduction to quantum computing and maths basics
2 – Single qubit representations and logic operations
3 – Two qubit states and logic gates
4– Multi-qubit states and quantum arithmetic
5 – Basic quantum algorithms
6 – Period finding, cryptography and quantum factoring
7– Shor's algorithm, post-quantum crypto, quantum key distribution
8 – Quantum search algorithms
9 – Grover search applications, optimisation problems
10 – Solving optimisation problems on quantum computers
11– Applications in quantum machine learning
12 – Real quantum computer devices

Assignment schedule:
#1: Hand out in Week 2
#2: Hand out in Week 8

Week 11

Lecture 21

1. Machine Learning
2. Neural Network
3. Support Vector Machine
4. Principal Component Analysis

Lecture 22

1. Quantum Machine Learning
2. Quantum Principal Component Analysis

Practice class 11

Quantum Principal Component Analysis

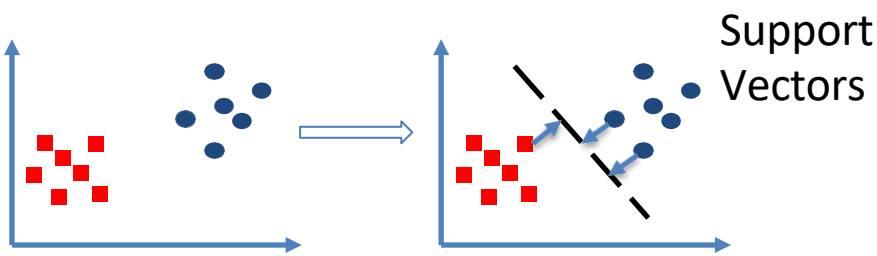
Recap: Lecture 21

"[Machine Learning is the] field of study that gives computers the ability to learn without being explicitly programmed."

Machine learning research has seen a rapid progress in recent years

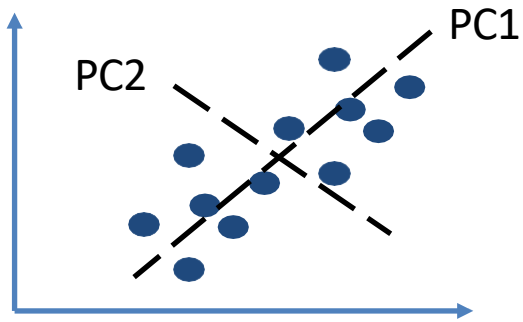
1. Generative AI (ChatGPT4, Midjourney etc.)
2. Programming (GitHub Copilot etc.)
3. Autonomous cars (Waymo in San Francisco, Baidu in Beijing's Yizhuang district, etc.)

Supervised Machine Learning



Support Vector Machine

Unsupervised Machine Learning

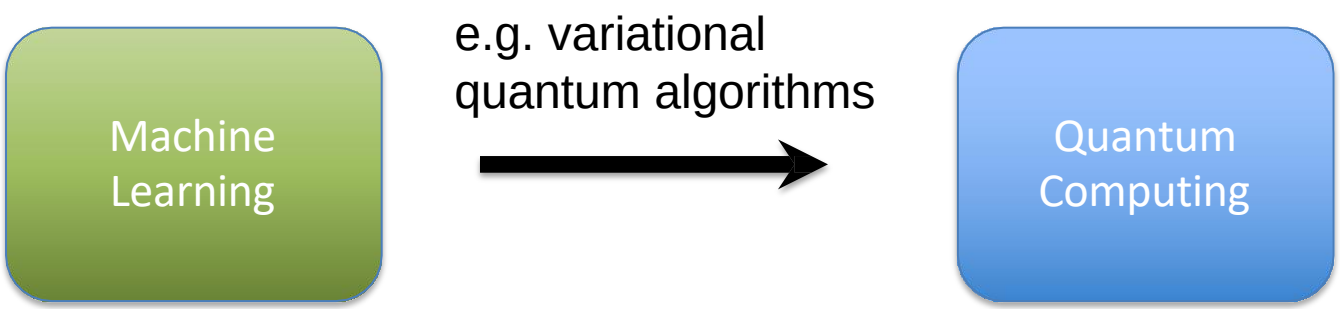


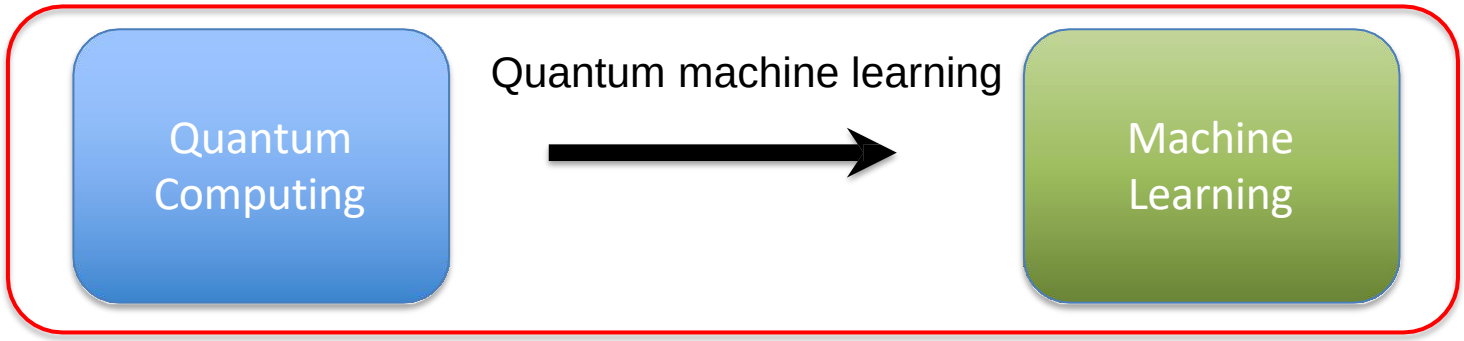
Principal Component Analysis

22.1 Quantum Machine Learning

Quantum Machine Learning Overview

- As mentioned in the last lecture, the intersection of machine learning and quantum computing has seen rapid progress in recent years.
- There are two clear directions in this cross-disciplinary research





The use of QC to improve ML

Machine learning algorithms commonly rely on linear algebra routines in very high dimensional vector spaces, and utilise routines such as as

- finding eigenstates / eigenvectors, and
- matrix inversion.

Quantum computing has the potential to provide algorithms for speed-ups of linear algebra tasks

- especially for the case when the matrices are *sparse*

Assume it takes $O(N)$bits to specify a classical vector.

- For a quantum computer we assume we have qubits: $N = 2^n$, *so* $n = \log_2 N$
- If a classical vector is *1* terabit=10^{12} bits, we would only need

$$\log_2 10^{12} \approx 40 \text{ qubits.}$$

The Quantum Algorithm Zoo outlines the algorithms and their *speed-ups*.

- https://quantumalgorithmzoo.org/

The use of QC to improve ML

In the quantum setting, the training data can possess *entanglement*

- *Quantum *no free lunch* theorem (can learn *more* from entangled training data)

- Even if training data does *not* posses entanglement (i.e. classical training data) quantum computing has the potential speed up linear algebra operations.

The quantum no free lunch theorem implies "*that one does not need an exponentially large training dataset in order to learn (unitary concepts*)"

- Hence, "*establishes that both big data and big entanglement are valuable in quantum machine learning, and that the currency of entanglement can lead to scalability*"

*Kunal Sharma et al. 2022, Reformulation of the No-Free-Lunch Theorem for Entangled Datasets, *Physical Review Letters*

Quantum Speed-ups for ML routines

How can quantum computing improve basic machine learning subroutines?

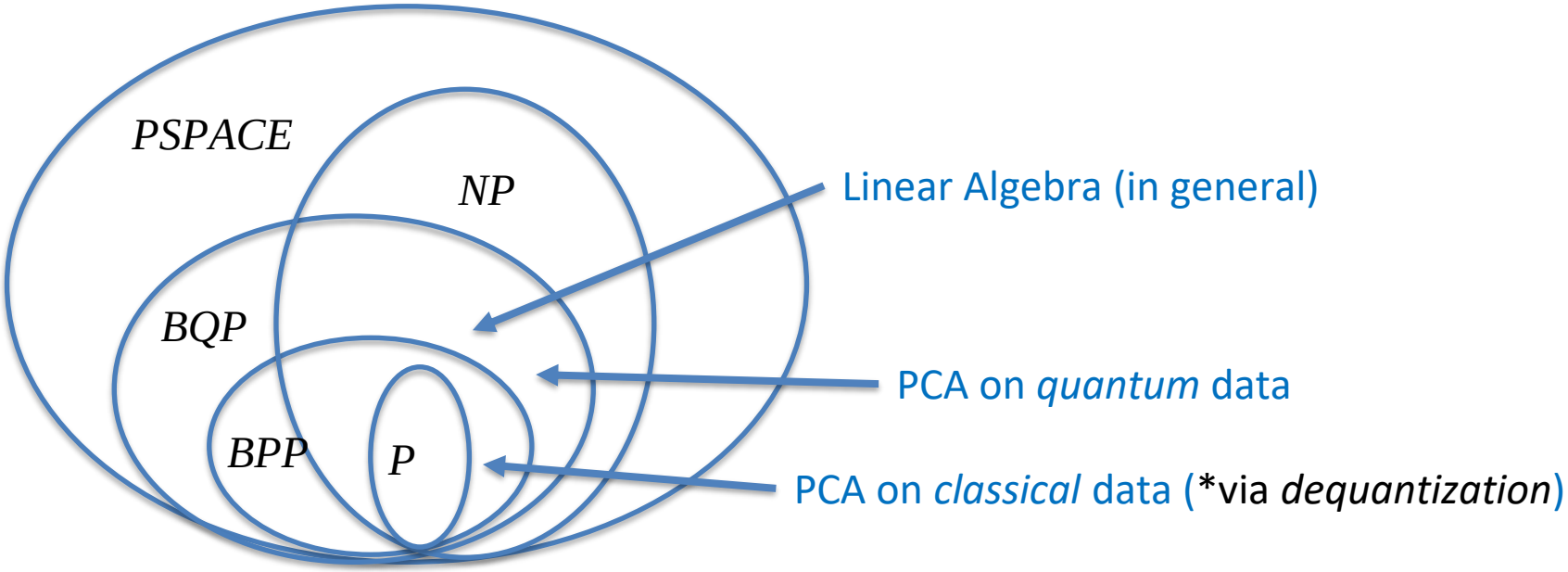
Method	Speed-up* (Classical input data) *Assumes error corrected	Speed-up* (Quantum input data)
Linear algebra (in general)	Super-polynomial *(e.g.* $2^{(\log n)^2}$*)*	Exponential (2^n, classical computers not suited to QC)
PCA	Polynomial (via *dequantization*)	Exponential
Grover search (applied to e.g. reinforcement learning)	Polynomial (e.g. $\sqrt{N}$ implies *quadratic* speedup over $O(N)$)	
Quantum Boltzmann machines	Super-polynomial	

*Not necessarily exponential speedup on classical data because it can take exponential time to prepare the input data for quantum computer

*Not necessarily exponential on NISQ computers, as *baron plateaus* can require exponential sampling to handle errors, although recent advances could overcome this.

The Quantum Algorithm Zoo outlines the algorithms and their *speed-ups (last updated June 2022)* https://quantumalgorithmzoo.org/

Complexity Classes (Recap)



BQP is Tractable on (error corrected) Quantum computers

P: Polynomial Time
BPP: Bounded-Error Probabilistic P Time
BQP: Bounded-Error Quantum P Time
NP: Nondeterministic P time

*Ewin Tang, 2022, Dequantizing algorithms to understand quantum advantage in machine learning, *Nature*

Scott Aaronson, 2015, Read the fine print, *Nature*

Figure taken from: An Introduction to Quantum computing by P. Kaye et.al., OXFORD Press

22.2 Quantum Principal Component Analysis

Quantum Principal Component Analysis (Revisited)

Recall in the last lecture we were introduced to principal component analysis (PCA), where the aim was to find the principal component(s) of a data set.

- The data are given by N-*dimensional* vectors $\vec{x}_j$.

- The covariance matrix is defined as $\Sigma = Cov(X,Y) = \frac{\sum_{i=1}^{n}(X_i - E[X])(Y_i - E[Y])}{n-1}$, where *E[X]* and *E[Y]* are the mean (or expected) values of the data along the *X* and *Y* dimensions. This summarises the correlations between the different components of the data.

In its simplest form, PCA operates by diagonalizing the covariance matrix, C:

$$C = \sum_k \lambda_k \vec{c}_k \vec{c}_k^T$$

Eigenvectors

Eigenvalues

- If only a few of the eigenvalues λ_k are large and the remainder small or zero, then the eigenvectors $\vec{c}_k$ corresponding to these eigenvalues are called the principal components.

*Quantum Principal Component Analysis (qPCA)

- To find the principal components with a quantum circuit, we *first* need to encode the covariance matrix in a quantum state *(quantum state preparation).*

  This is done by encoding the data vectors $\vec{x}_j$ as a superposition of states:

$$\vec{x}_j \rightarrow |x_j\rangle = \sum_i x_j^i |i\rangle$$

  We chose states, $|x_j\rangle$, *at random* to encode the covariance matrix.

- *Second*, we prepare many copies of each randomly chosen state, $|x_j\rangle$, in order to effectively have many copies of the covariance matrix.

- *Third*, we perform a natural exponential *SWAP* operation (SWAP seen in earlier lectures) on *each* copy of the covariance matrix state (based on *Euler's* formula):

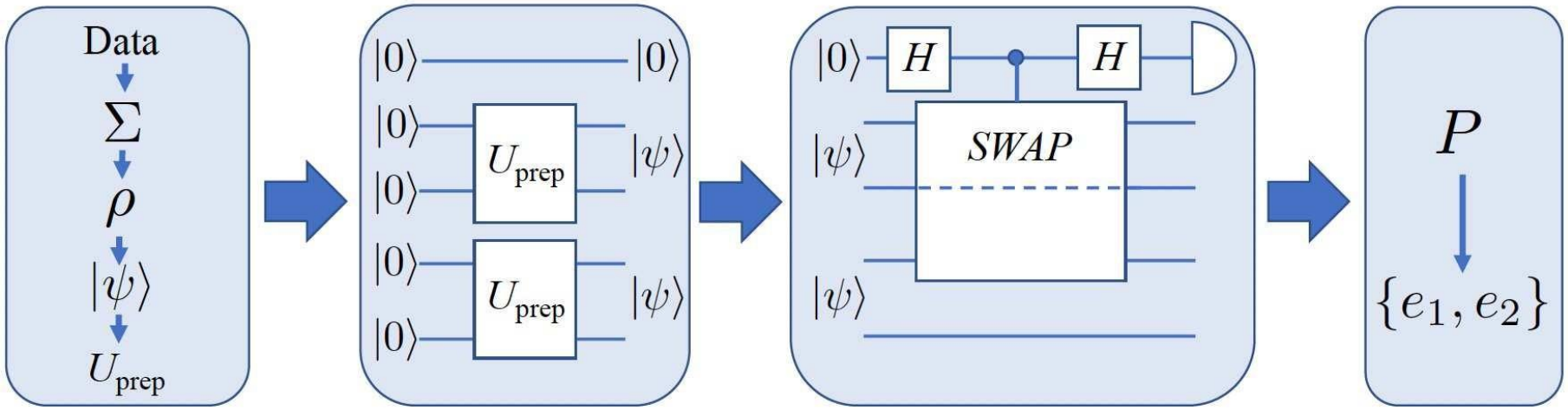
$$e^{-i\ \mathrm{SWAP}\Delta t} = \cos(\Delta t)\mathbb{I} - i\sin(\Delta t)\ \mathrm{SWAP}$$

- *Fourth*, we use the quantum *phase estimation* algorithm to determine the eigenvalues.

*Seth Lloyd et al. 2014 Quantum principal component analysis, Nature Physics

Quantum Principal Component Analysis (qPCA)

In the special case of only two features (so a *2x2* covariance matrix), this procedure can be simplified as:



Quantum state preparation

Classical post-processing

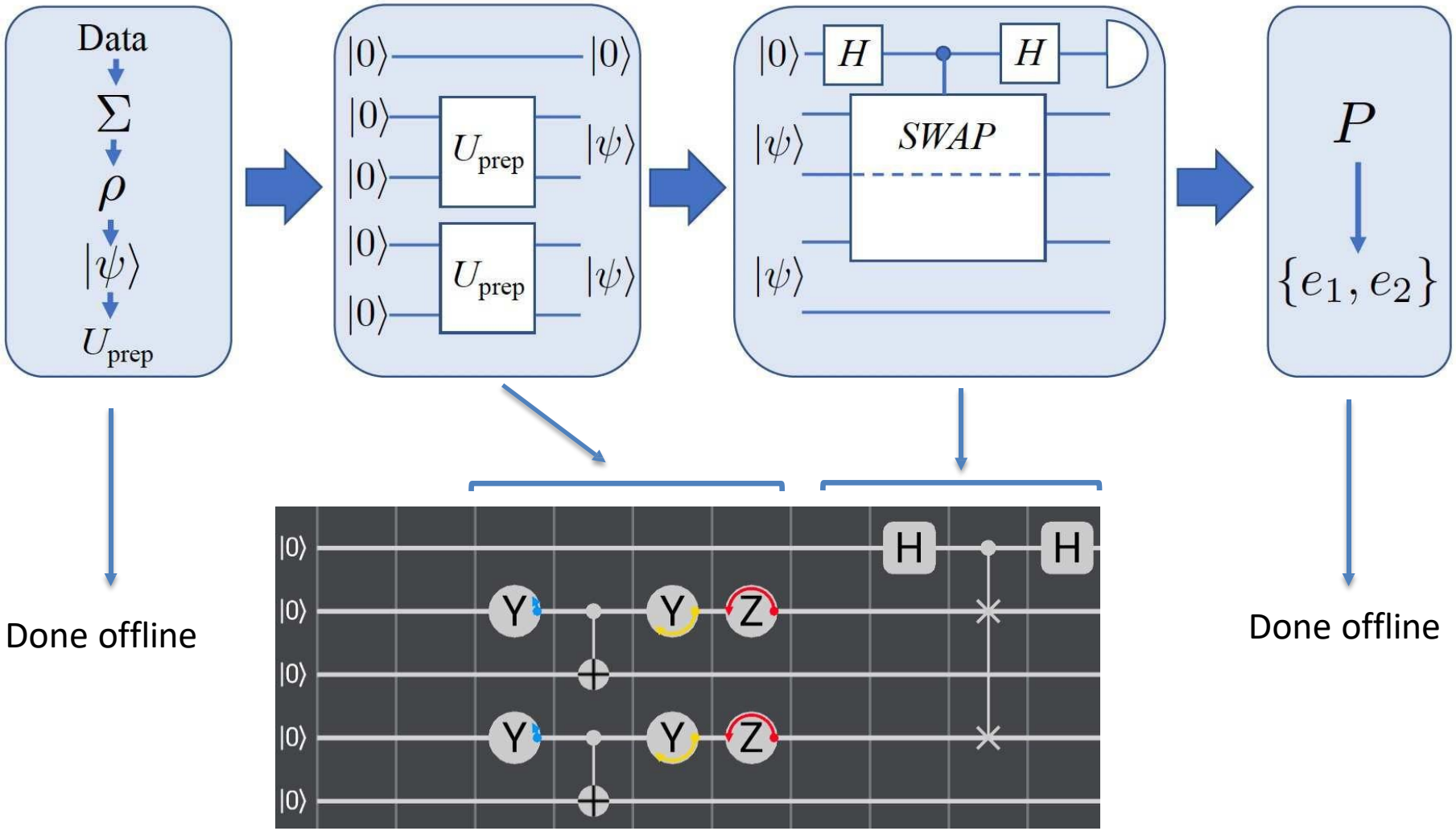
Classical pre-processing, where we calculate the covariance matrix.

(Simplified) exponential SWAP

Σ is the covariance matrix, $\rho = \frac{Z}{Tr[Z]}$, P is the difference in probability for measuring a *0* or *1* on the first qubit and $e_1$and e_2 are the estimated *eigenvalues*.

Quantum Principal Component Analysis (qPCA)

In the special case of only two features (so a *2x2* covariance matrix), this procedure can be simplified as:



qPCA example – classical formulation

Suppose we are given data for hotel prices based on number of bedrooms as follows:

$\boldsymbol{X}$ = Number of rooms = *{1, 2, 2, 3, 1, 1, 1, 2, 1, 3, 3, 4, 1, 2, 2}*

$\boldsymbol{Y}$ = Price per night = *{102, 156, 192, 254, 131, 121, 141, 163, 287, 356, 96, 144, 121, 306, 139}*

For scaling purpose, we divide prices by *100*:

$\boldsymbol{Y}$ = Price per night = *{1.02, 1.56, 1.92, 2.54, 1.31, 1.21, 1.41, 1.63, 2.87, 3.56, 0.96, 1.44, 1.21, 3.06, 1.39}*

$$Cov(X,Y) = \frac{\sum_{i=1}^{n}(X_i - E[X])(Y_i - E[Y])}{n-1} = \begin{pmatrix} Cov(X,X) & Cov(X,Y) \\ Cov(Y,X) & Cov(Y,Y) \end{pmatrix}$$

$E[X] = 1.93$

$E[Y] = 1.806$

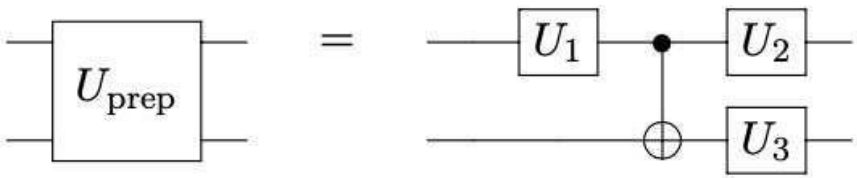
$$Cov(X,Y) = \begin{bmatrix} 0.9238 & 0.194 \\ 0.194 & 0.6556 \end{bmatrix} \rightarrow \text{Eigen Values of Covariance} = [0.5539\ 1.0255]$$

qPCA example – quantum formulation

In order to load the covariance data into a quantum circuit, first we need to compute normalised covariance matrix, ρ, (also called *density* matrix) as follows:

$$\rho = \frac{Covariance}{Trace[Covariance]} = \begin{bmatrix} 0.5849 & 0.1228 \\ 0.1228 & 0.4151 \end{bmatrix}$$ where, *Trace(Cov(X,Y)) = 1.5794*

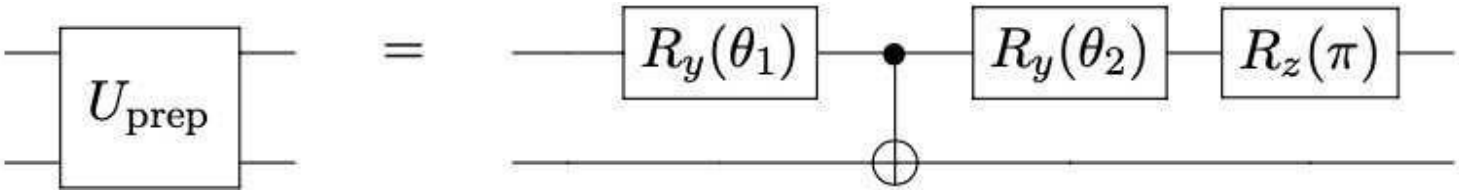
Based on the ρ above, we will prepare a quantum circuit as follows:



where the unitaries U_1, U_2 and U_3 are general single qubit unitaries and are of the form:

$U_i = R_z(\phi_i)R_y(\theta_i)R_z(\sigma_i)$, with *i = 1, 2 or 3.*

qPCA example – quantum



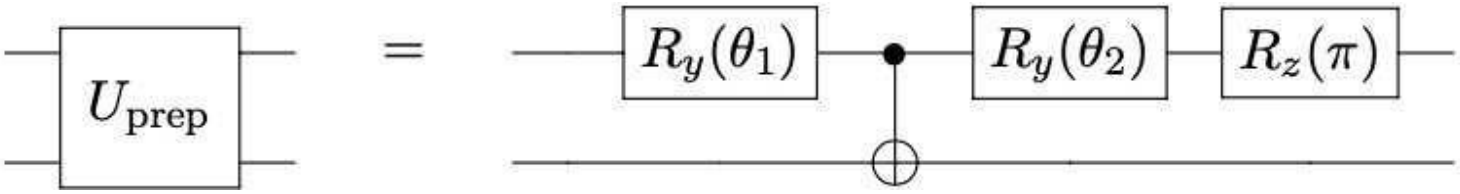
$$\rho = \begin{bmatrix} 0.5849 & 0.1228 \\ 0.1228 & 0.4151 \end{bmatrix} = \begin{pmatrix} \frac{1}{2}(1 + Cos(\theta_1)\, Cos(\theta_2)) & -\frac{1}{2} Cos(\theta_1)\, Sin(\theta_2) \\ -\frac{1}{2} Cos(\theta_1)\, Sin(\theta_2) & \frac{1}{2}(1 - Cos(\theta_1)\, Cos(\theta_2)) \end{pmatrix}$$

By solving the equation above, we get the following values for θ_1 and θ_2

$$\theta_1 = 1.2676$$

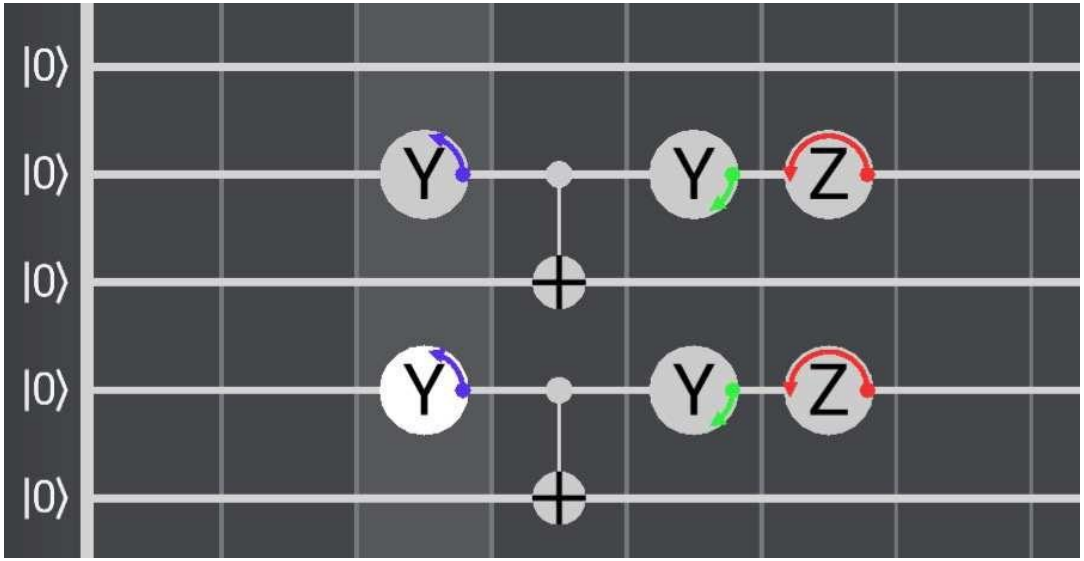
$$\theta_2 = -0.9659$$

qPCA example – quantum

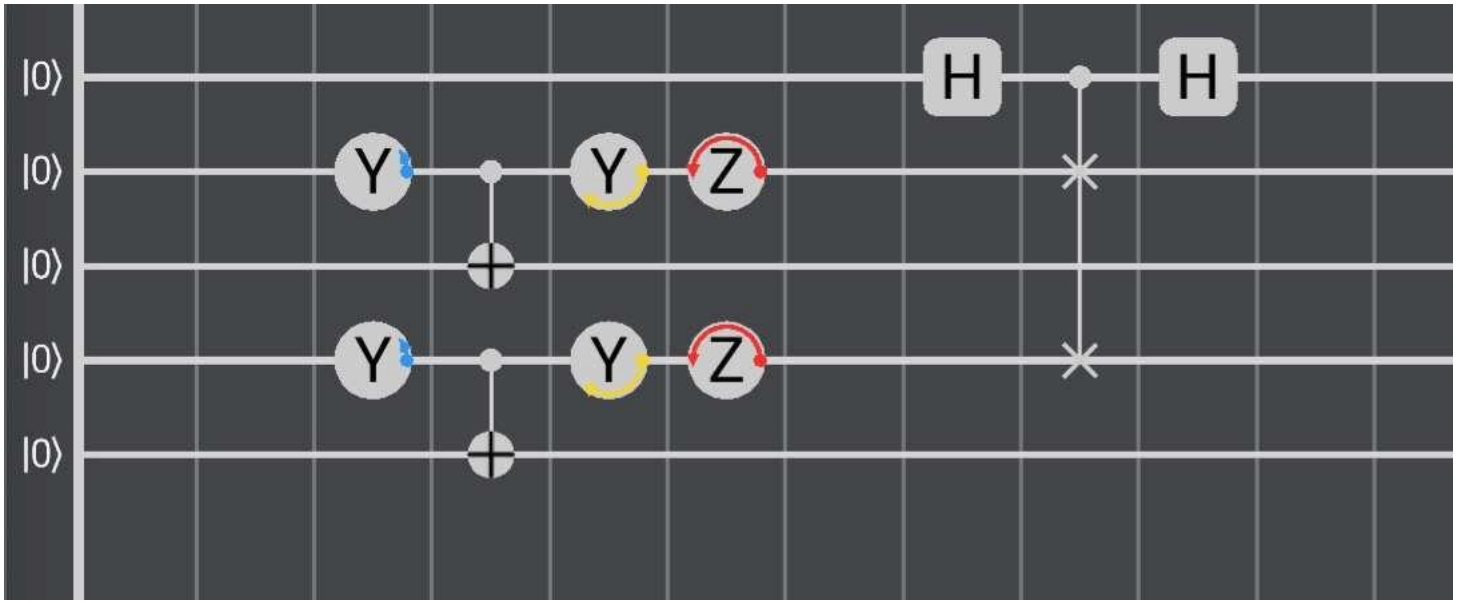


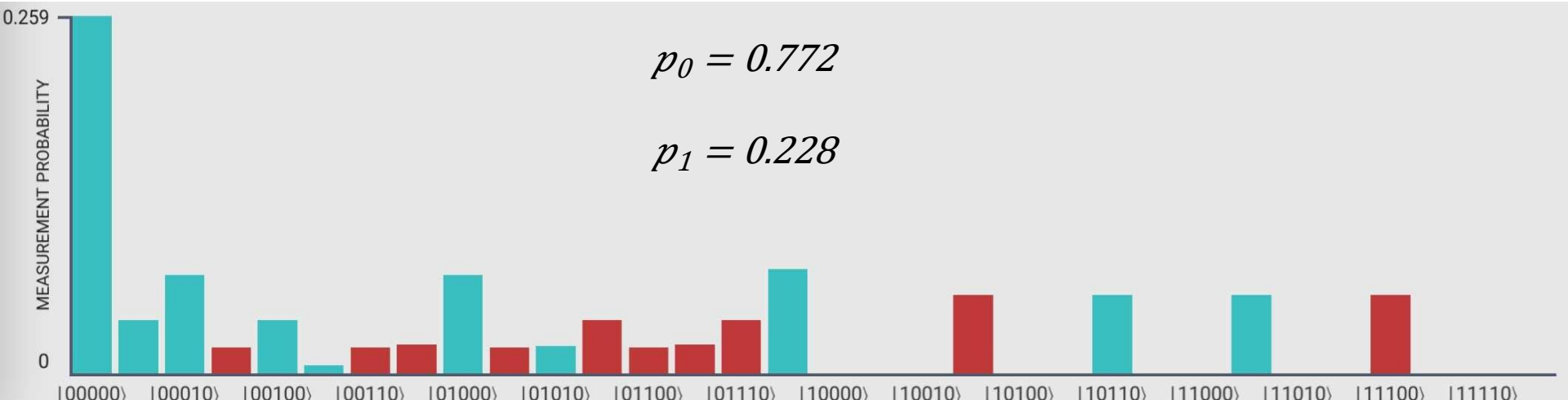
$$\theta_1 = 1.2676$$

$$\theta_2 = -0.9659$$



qPCA example – quantum





$p_0 = 0.772$

$p_1 = 0.228$

Working through a *qPCA* example

$p_0 = 0.772$

$p_1 = 0.228$

The estimated eigenvalues for this the Covariance matrix are consequently computed (offline by classical computer) by:

$$\lambda_1 = Trace[Covariance]\frac{\left(1+\sqrt{1-2(1-P)}\right)}{2}$$

$$\lambda_2 = Trace[Covariance]\frac{\left(1-\sqrt{1-2(1-P)}\right)}{2}$$

Recall:
Trace*(Cov(X,Y)) = 1.5794*

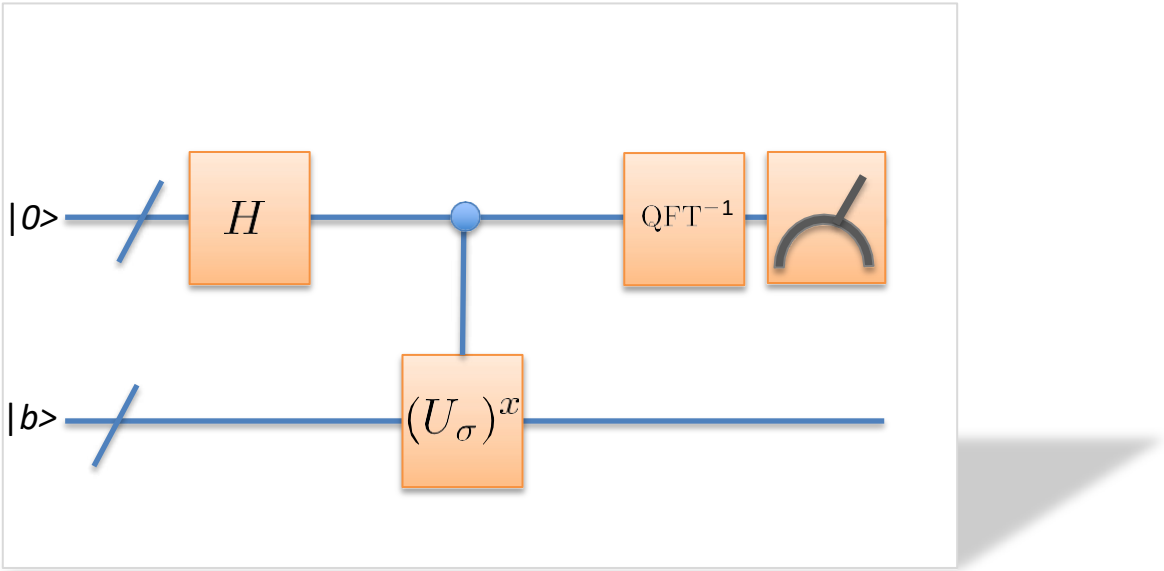
$P = p_0 - p_1 0.544 = 0.228 - 0.772 =$

$\lambda_1 = 1.024$
$\lambda_2 = 0.556$

Cross-check - recall that the Eigen Values of Covariance = *[0.5539 1.0255]*

Quantum Principal Component Analysis (qPCA)

Another method to perform principal component analysis that is better adapted to noisy intermediate-scale quantum devices is based solely on the quantum phase estimation algorithm:



General *n*n* case:

- The unitary matrix U_σ is function of the covariance matrix: $U_\sigma = \exp\left(2\pi i \frac{\sigma}{\mathrm{tr}(\sigma)}\right)$
- The top register measurement gives the maximum eigenvalue of the covariance matrix.

Experimental qPCA on NISQ

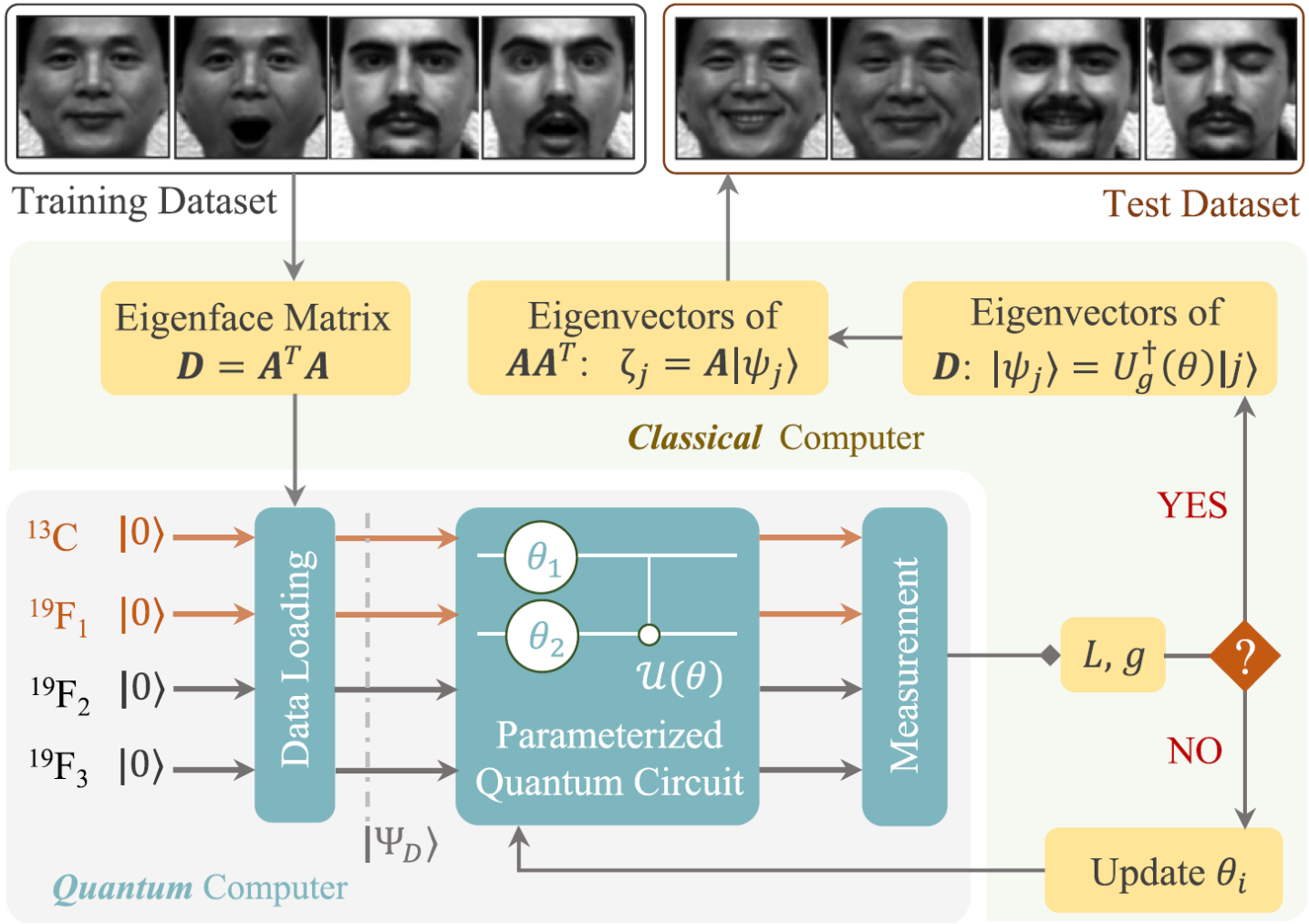
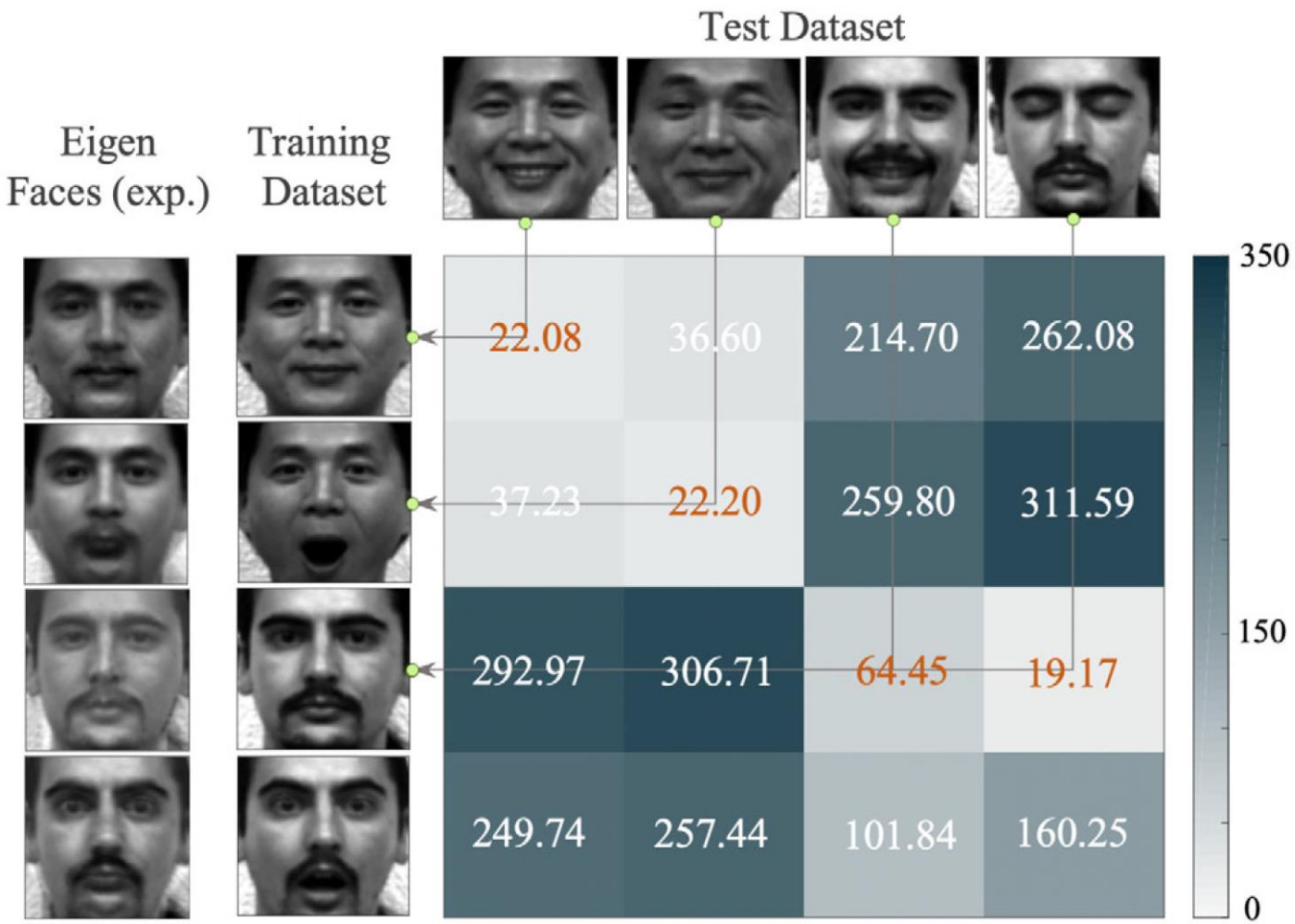


Figure from Tao Xin et al. 2021 Experimental Quantum Principal Component Analysis via Parametrized Quantum Circuits, *Physical Review Letters*

Experimental qPCA on NISQ



Quantum circuit for NLP outperforms classical state of the art

*Aspect-based sentiment analysis (ABSA) for translation using quantum circuit

- Outperforms the state of the art for classical algorithms on ABSA

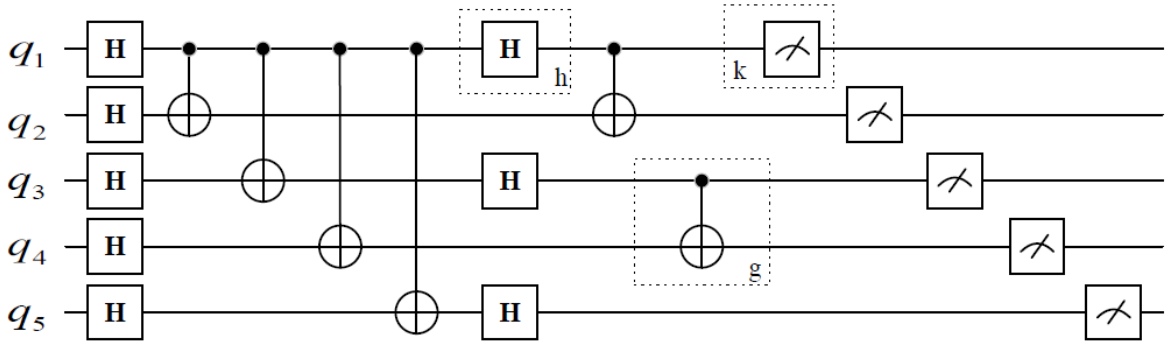


Figure 4: Quantum circuit implementation of the quantum entanglement module, where h, g, k denote the Hadamard gate, the CNOT gate, the measure gate, respectively.

*Zhao et al. 2024 QPEN: Quantum Projection and Quantum Entanglement Enhanced Network for Cross-Lingual Aspect-Based Sentiment Analysis, *AAAI 2024*

Hot topic: Variational Quantum Algorithms (for NISQ)

*Variational quantum algorithms (VQAs) use a classical optimizer to train a parameterized quantum circuit.

They address NISQ (noisy intermediate-scale quantum computer) constraints, including

- limited numbers of qubits, and
- noise processes that limit circuit depth.

Techniques to overcome the problem of barren plateaus in VQAs are emerging**

VQAs + error correction is an emerging approach***

*M. Cerezo et al. 2021 Variational quantum algorithms, *Nature*

**Heredge 2024, Non-Unitary Quantum Machine Learning

**Fontana 2024, Characterizing barren plateaus in quantum ansätze with the adjoint representation

***Campbell 2023, A series of fast-paced advances in Quantum Error Correction, *Nature Reviews Physics*

***Babbush 2024, Error corrected variational algorithms (Patent)

Week 11

Lecture 21

1. Machine Learning
2. Neural Network
3. Support Vector Machine
4. Principal Component Analysis

Lecture 22

1. Quantum Machine Learning
2. Quantum Principal Component Analysis

Practice class 11

Quantum Machine Learning

Subject outline

Lecture topics (by week)

1 – Introduction to quantum computing and maths basics
2 – Single qubit representations and logic operations
3 – Two qubit states and logic gates
4– Multi-qubit states and quantum arithmetic
5 – Basic quantum algorithms
6 – Period finding, cryptography and quantum factoring
7– Shor's algorithm, post-quantum crypto, quantum key distribution
8 – Quantum search algorithms
9 – Grover search applications, optimisation problems
10 – Solving optimisation problems on quantum computers
11– Applications in quantum machine learning
12 – Real quantum computer devices

Assignment schedule:
#1: Hand out in Week 2
#2: Hand out in Week 8